SISTEMAS OPERATIVOS

Aula Teórica 12

Daniel Castro
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

Deadlocks (1)

1. Resource
2. Introduction to deadlocks
3. The ostrich algorithm
4. Deadlock detection and recovery
5. Deadlock avoidance
6. Deadlock prevention
7. Other issues

Resources (1)

A **Resource** can be a hardware device (ex. a printer) or a piece of information (ex. a record in a database).

Examples of computer resources:
- Memory space - I/O devices - SSD -Tables - Printers - CPU cycles

In short, a resource is anything that must be acquired, used, and released over the course of time.

Preemptable Resource : is a resource that can be taken away from the process owning it with no ill effect;

Example: Memory. Consider a system with 16GB of user memory, one printer, and two 16GB processes, that each one wants to print something.

1. Process *A* request and gets the printer, then starts to computre the values to print. Before finishing the computation, it exceeds its time quantum and is swapped out tyo SSD or disk.
2. Process *B* now runs and tries, unsuccefully to acquire the printer. Potentially we have a deadlock, because *A* has the printer, *B* has the memory, and neither one can proceed without the resource held by the other.
Fortunately, it is possible to preempt (take away) the memory from *B* by swapping it out and swapping *A* in.
3. Now *A* can run, do its printing, and then release the printer. No deadlock occurs.

Nonpreemtable Resource : is a resource that cannot be taken away from its current owner without potentially causing failure.
Example: 3D printer.

Resources (2)

The sequence of events required to use a resource is given below:

1. request the resource;
2. use the resource;
3. release the resource.

If the resource is not available when it is requested, the requesting process is forced to wait.
In some OS, the process is automatically blocked when a resource request fails, and awakened when it is available.
In other systems, the request fails with an error code, and it is up to the calling process to wait a little while and try again.

The exact nature of requesting a resource is highly system dependent.

In some systems, a request system call is provided to allow a process to explicitly ask for a resource.

In others, the only resource that the operating system knows about are special files that only one process can have open at a time. If the file is already in use, the caller is blocked until its current owner close it.

Resources (3)

Resources acquisition

For some kinds of resources, like records in a Database system, it is up to the user process the ability to manage resources usage themselves.

One way to do this
is associate a semaphore to each resource.
Initialized the semaphore as 1,
and do:

1. down(resource_semaphore);
2. use_the_resource();
3. up(resource_semaphore);

```
typedef int semaphore;
semaphore resource_1;

void process_A(void) {
      down(&resource_1);
      use_resource_1( );
      up(&resource_1);
}
```

(a)

```
typedef int semaphore;
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
      down(&resource_1);
      down(&resource_2);
      use_both_resources( );
      up(&resource_2);
      up(&resource_1);
}
```

(b)

Figure 6-1. Using a semaphore to protect resources. (a) One resource. (b) Two resources.

Deadlock (2)

Conditions for **Resource Deadlocks**:

1. Mutual exclusion – Each resource is either currently assigned to exactly one process or is available;
2. Hold-and-wait – Processes currently holding resources that were granted earlier can request a resource;
3. No-preemption – Resources previously granted cannot be forcibly taken away from a process. They must be explicitly released by the process holding them;
4. Circular wait – There must be a circular list of two or more processes, each of which is waiting for a resource held by the next member of the chain.

All of these conditions must be present for a resource deadlock to occur.

Deadlock Modeling

The four conditions above, could be modeled using directed graphs. Processes - ○ Resource - □

(a) A directed arc from a resource to a process means that the resource *R* is currently assigned to process *A*.

(b) The process *B* is currently blocked waiting for resource *S*.

(c) Deadlock. Process *C* is waitng for *T*, which is held by *D*, Process *D* is not about to release *T* because is waiting for *U*, held by *C*. Both processes will wait forever.

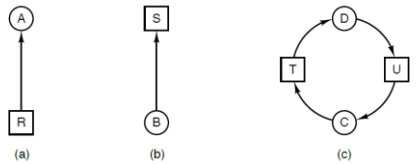


Figure 6-6. Resource -allocation graphs. (a) Holding a resource. (b) Requesting a resource. (c) Deadlock.

Deadlock (3)

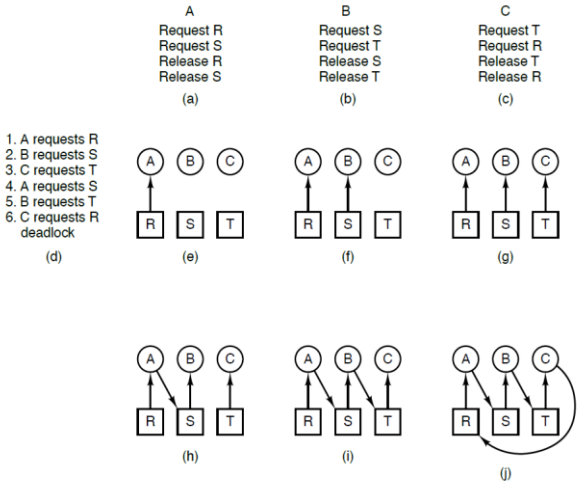
Example

We have 3 processes *A*, *B*, *C*, and 3 resources *R*, *S*, *T*. The Request and Release order of the three processes are listed in (a)(b)(c).

The OS is free to run any unblocked process at any instant.
So it could decide to run *A* until *A* finished all its work, then run *B* to completion, and finally run *C*.
But this kind of running, strictly sequentially, may not be optimal due to all processes do both I/O and computing. In this case, round-robin is a reasonable scheduling algorithm.

(d) Requests order.

>>> Deadlock occurs.



Deadlock (4)

Example

The OS is under no obligation to run the process in any particular order.

If granting a particular request might lead to deadlock, the OS can simply suspend the process without granting the request, until it is safe.

If the OS knew about the impending deadlock, it could suspend *B* without granting the request *S*.

By running only *A* and *C*, the deadlock can be avoided.

(k) New request/release order.

>>> Deadlock can be avoided.

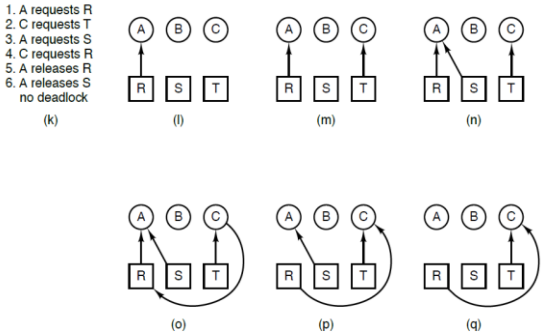


Figure 6-7. An example of how deadlock occurs and how it can be avoided.

Strategies for dealing with Deadlocks

1. Just ignore the problem. May be if you ignore it, it will ignore you.

2. Detection and recovery. Let them occur, detect them, and take action.

3. Dynamic avoidance by careful resource allocation.

4. Prevention, by structurally negating one of the four necessary conditions.

The Ostrich Algorithm (1)

The Ostrich Algorithm

Pretend there is no problem.

Reasonable if
- deadlock occurs very rarely
- cost of prevention is high

Unix and Windows takes this approach
It is a trade off between
- convenience
- correctness



Avestruz no Badoca Safari Park

Example:
Consider an OS that blocks the caller when an *open* system call on a physical device such as a 3D scanner or a printer cannot be carried out because the device is busy. Typically it is up to the device driver to decide what action to take under such circunstances. Blocking or returning an error code are two obvious possibilities.
If one process successfully opens the scanner and another successfully opens the printer and then each process tries to open the other one and blocks trying, we have a deadlock. Few current systems will detect this.

Detection with One Resource of Each Type (1)

A second technique is **detection and recovery**. When this technique is used, the system does not attempt to prevent deadlocks from occuring. Instead, it lets them occur, tries to detect when this happens, and then takes some action to recover after the fact.

Example:
There is only one resource of each type and each device can be acquired by a single process.
7 processes *A*-*G*, 6 resources: Blu-ray recorder – *R*, Scanner – *S*, Tape drive – *T*, USB microphone – *U*, Video camera – *V*, Waffer cutter – *W*.

1. Process *A* holds *R* and wants *S*.
2. Process *B* holds nothing but wants *T*.
3. Process *C* holds nothing but wants *S*.
4. Process *D* hold *U* and wants *S* and *T*.
5. Process *E* holds *T* and wants *V*.
6. Process *F* holds *W* and wants *S*
7. Process *G* holds *V* and wants *U*.

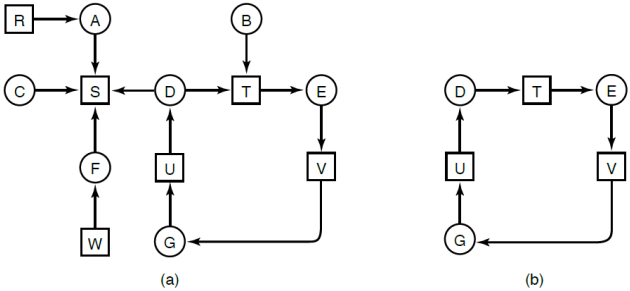


Figure 6-8. (a) A resource graph. (b) A cycle extracted from (a).

Detection with One Resource of Each Type (2)

If the graph contains no cycles ⇨ no deadlock.

If the graph contains one or more cycles ⇨ deadlock exists.

Although it is relatively simple to pick out the deadlocked processes by visual inspection from a single graph, where we can see that processes *D*, *E*, and *G* are all deadlocked, for use in actual systems we need a formal algorithm for detecting deadlocks.

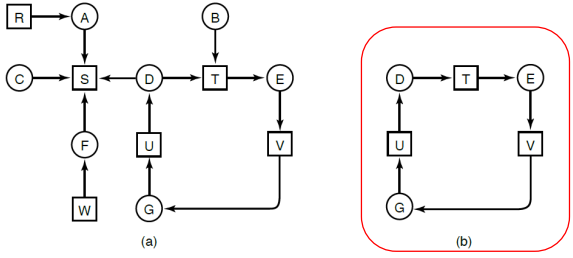


Figure 6-8. (a) A resource graph. (b) A cycle extracted from (a).

Detection with One Resource of Each Type (3)

Algorithm
It uses a dynamic data struture *L*, a list of nodes, as well as a list of arcs.

1. For each node *N*, in the graph, perform the following five steps with *N* as the starting node.

   2. Initialize *L* to empty list, and designate all the arcs as unmarked.

   3. Add the current node to the end of *L* and check to see if the node now appears in *L* two times. If it does, the graph contains a cycle (listed in *L*) and the algorithm terminates.

   4. From the given node, see if there are any unmarked outgoing arcs.
      If so go to step 5, if not go to setp 6.

      5. Pick an unmarked outgoing arc at random and mark it. Then follow it to the new current node and go to step 3.

      6. If this node is the initial node, the graph does not contain any cycles and the algorithm terminates.
         Otherwise, we have now reached a dead end. Remove it and go back to the previous node, that is the one that was current just before this one, make that one the current node, and go to step 3

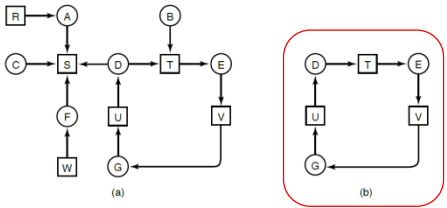


Figure 6-8. (a) A resource graph. (b) A cycle extracted from (a).

Detection with One Resource of Each Type (4)

The order of processing the nodes is arbitrary, so let us just inspect them from left to right, top to bottom, first running the algorithm starting at *R*, then successively *A*, *B*, *C*, *S*, *D*, *T*, *E*, *F*, …

Node R: []
[R]
[R, A]
[R, A, S]
There are no outgoing arcs at S, so it is a dead end.

Node A: []
[A]
[A, S] -> dead end.

Node B: []
[B]
[B, T] ….
[B,T,E,V,G,U, D] we have two outgoing arcs, so we must make a (random) choice
if we pick S -> dead end
if we pick T -> [B,T,E,V,G,U,D,T] -> we discover a cycle
and the algorithm stops.

1. For each node *N*, in the graph, perform the following five steps with *N* as the starting node.
2. Initialize *L* to empty list, and designate all the arcs as unmarked.
3. Add the current node to the end of *L* and check to see if the node now appears in *L* two times. If it does, the graph contains a cycle (listed in *L*) and the algorithm terminates.
4. From the given node, see if there are any unmarked outgoing arcs. If so go to step 5, if not go to setp 6.
5. Pick an unmarked outgoing arc at random and mark it. Then follow it to the new current node and go to step 3.
6. If this node is the initial node, the graph does not contain any cycles and the algorithm terminates. Otherwise, we have now reached a dead end. Remove it and go back to the previous node, that is the one that was current just before this one, make that one the current node, and go to step 3

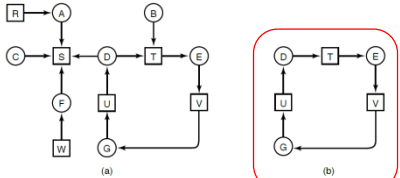


Figure 6-8. (a) A resource graph. (b) A cycle extracted from (a).

This algorithm is far from optimal, nevertheless it demonstrates that an algorithm for deadlock detection exists.

QUESTÕES?

Daniel Castro
dcastro@ualg.pt

